🤖 NewsBot Intelligence System

ITAI 2373 - Mid-Term Group Project Template

Team Members: [Add your names here] **Date:** [Add date] **GitHub Repository:** [Add your repo URL here]

🎯 Project Overview

Welcome to your NewsBot Intelligence System! This notebook will guide you through building a comprehensive NLP system that:

- 📰 **Processes** news articles with advanced text cleaning
- 🏷️ **Classifies** articles into categories (Politics, Sports, Technology, Business, Entertainment, Health)
- 🔍 **Extracts** named entities (people, organizations, locations, dates, money)
- 😊 **Analyzes** sentiment and emotional tone
- 📊 **Generates** insights for business intelligence

📚 Module Integration Checklist

- [ ] **Module 1:** NLP applications and real-world context
- [ ] **Module 2:** Text preprocessing pipeline
- [ ] **Module 3:** TF-IDF feature extraction
- [ ] **Module 4:** POS tagging analysis
- [ ] **Module 5:** Syntax parsing and semantic analysis
- [ ] **Module 6:** Sentiment and emotion analysis
- [ ] **Module 7:** Text classification system
- [ ] **Module 8:** Named Entity Recognition

```
import pandas as pd

# Load the dataset
df = pd.read_csv("BBC News Train.csv")

# Basic information
print("Dataset shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nCategory distribution:")
print(df["Category"].value_counts())

# Display first rows
df.head()
```

```
Dataset shape: (1490, 3)

Columns:
Index(['ArticleId', 'Text', 'Category'], dtype='object')

Category distribution:
Category
sport            346
business         336
politics         274
entertainment    273
tech             261
Name: count, dtype: int64
```

	ArticleId	Text	Category
0	1833	worldcom ex-boss launches defence lawyers defe...	business
1	154	german business confidence slides german busin...	business
2	1101	bbc poll indicates economic gloom citizens in ...	business
3	1976	lifestyle governs mobile choice faster bett...	tech
4	917	enron bosses in $168m payout eighteen former e...	business

Next steps: Generate code with df New interactive sheet

```
print(df.isnull().sum())
```

```
ArticleId    0
Text         0
Category     0
dtype: int64
```

📦 Setup and Installation

Let's start by installing and importing all the libraries we'll need for our NewsBot system.

```
# Install required packages (run this cell first!)
!pip install spacy scikit-learn nltk pandas matplotlib seaborn wordcloud plotly
!python -m spacy download en_core_web_sm

# Download NLTK data
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('vader_lexicon')
nltk.download('averaged_perceptron_tagger')

print("✅ All packages installed successfully!")
```

```
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from ma
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from py
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from h
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from 
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 82.7 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
✅ All packages installed successfully!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip
```

```python
# Import all necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import plotly.express as px
import plotly.graph_objects as go
from collections import Counter, defaultdict
```

```
import re
import warnings
warnings.filterwarnings('ignore')

# NLP Libraries
import spacy
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.sentiment import SentimentIntensityAnalyzer
from nltk.tag import pos_tag

# Scikit-learn for machine learning
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.pipeline import Pipeline

# Load spaCy model
nlp = spacy.load('en_core_web_sm')

# Set up plotting style
plt.style.use('default')
sns.set_palette("husl")

print("📚 All libraries imported successfully!")
print(f"🔧 spaCy model loaded: {nlp.meta['name']} v{nlp.meta['version']}")
```

```
📚 All libraries imported successfully!
🔧 spaCy model loaded: core_web_sm v3.8.0
```

📊 Data Loading and Exploration

🎯 Module 1: Understanding Our NLP Application

Before we dive into the technical implementation, let's understand the real-world context of our NewsBot Intelligence System. This system addresses several business needs:

1. **Media Monitoring:** Automatically categorize and track news coverage
2. **Business Intelligence:** Extract key entities and sentiment trends
3. **Content Management:** Organize large volumes of news content
4. **Market Research:** Understand public sentiment about topics and entities

💡 **Discussion Question:** What other real-world applications can you think of for this type of system? Consider different industries and use cases.

```
# Load the BBC News dataset
import pandas as pd

# Read the CSV file uploaded to Colab
```

```
df = pd.read_csv("BBC News Train.csv")

# Rename columns to match the notebook's expected format
df = df.rename(columns={
    "ArticleId": "article_id",
    "Text": "content",
    "Category": "category"
})

# Create a title column from the first few words of the article
df["title"] = df["content"].str.split().str[:8].str.join("...")

# Add placeholder columns expected by the notebook
df["date"] = "2024-01-15"
df["source"] = "BBC News"

# Rearrange columns
df = df[["article_id", "title", "content", "category", "date", "source"]]

print("📊 Dataset loaded successfully!")
print(f"📈 Shape: {df.shape}")
print(f"📋 Columns: {list(df.columns)}")

# Display first few rows
df.head()
```

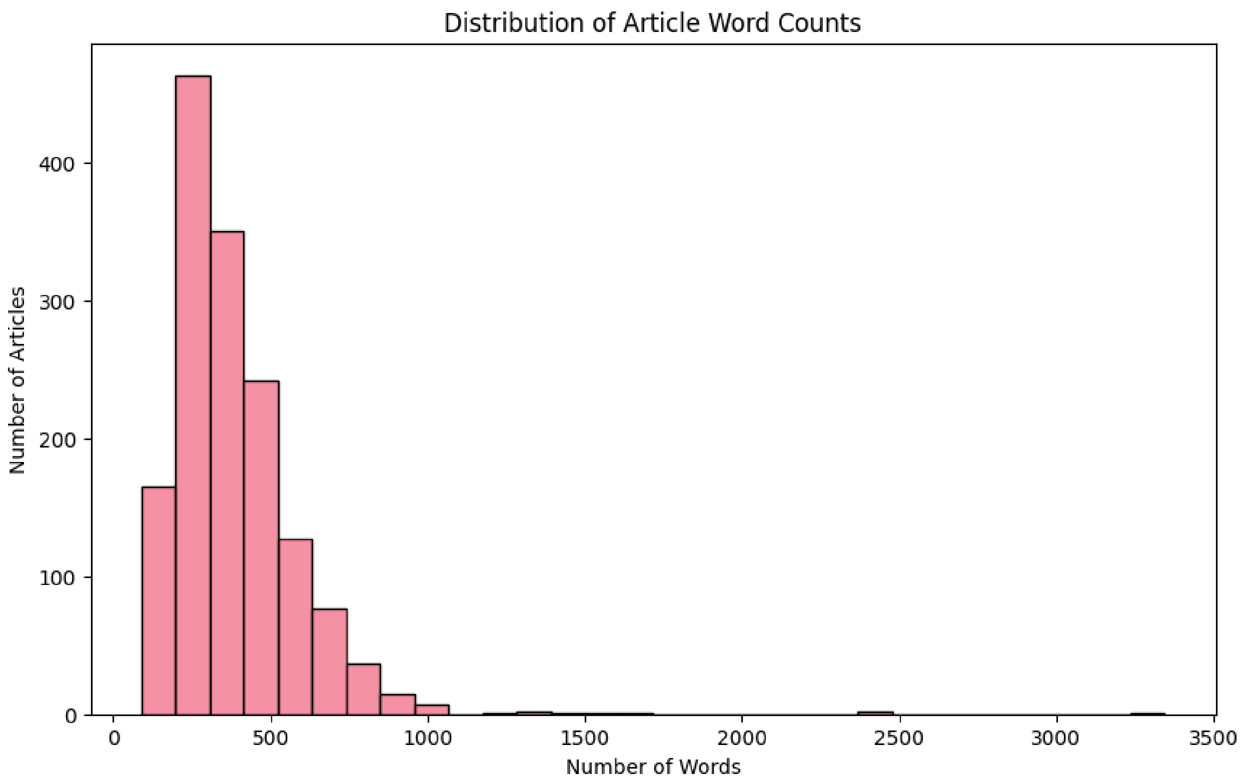
```
📊 Dataset loaded successfully!
📈 Shape: (1490, 6)
📋 Columns: ['article_id', 'title', 'content', 'category', 'date', 'source']
```

	article_id	title	content	category	date	source
0	1833	worldcom...ex-boss...launches...defence...lawy...	worldcom ex-boss launches defence lawyers defe...	business	2024-01-15	BBC News
1	154	german...business...confidence...slides...germ...	german business confidence slides german busin...	business	2024-01-15	BBC News
2	1101	bbc...poll...indicates...economic...gloom...ci...	bbc poll indicates economic gloom citizens in ...	business	2024-01-15	BBC News
3	1976	lifestyle...governs...mobile...choice...faster...	lifestyle governs mobile choice faster bett...	tech	2024-01-15	BBC News
			enron			

Next steps: Generate code with df New interactive sheet

```
import matplotlib.pyplot as plt
import seaborn as sns

# Check for missing values
print("\n🔍 MISSING VALUES")
print("=" * 50)
print(df.isnull().sum())

# Analyze article length
df['text_length'] = df['content'].str.len()
df['word_count'] = df['content'].str.split().str.len()

print("\n📝 ARTICLE LENGTH STATISTICS")
print("=" * 50)
print(df['word_count'].describe())

# Visualize word counts
plt.figure(figsize=(10,6))
sns.histplot(df['word_count'], bins=30)
plt.title('Distribution of Article Word Counts')
plt.xlabel('Number of Words')
plt.ylabel('Number of Articles')
plt.show()

# Source distribution
print("\n📰 SOURCE DISTRIBUTION")
print("=" * 50)
print(df['source'].value_counts())

# Data quality checks
print("\n✅ DATA QUALITY CHECK")
print("=" * 50)
print(f"Duplicate articles: {df.duplicated(subset=['content']).sum()}")
print(f"Empty articles: {(df['content'].str.len() == 0).sum()}")
print(f"Shortest article: {df['word_count'].min()} words")
print(f"Longest article: {df['word_count'].max()} words")
```

```
🔍 MISSING VALUES
==================================================
article_id    0
title         0
content       0
category      0
date          0
source        0
dtype: int64

📝 ARTICLE LENGTH STATISTICS
==================================================
count    1490.000000
mean      385.012752
std       210.898616
min        90.000000
25%       253.000000
50%       337.000000
75%       468.750000
max      3345.000000
Name: word_count, dtype: float64
```




```
📰 SOURCE DISTRIBUTION
==================================================
source
BBC News    1490
Name: count, dtype: int64

✅ DATA QUALITY CHECK
==================================================
Duplicate articles: 50
Empty articles: 0
Shortest article: 90 words
Longest article: 3345 words
```

🧹 Text Preprocessing Pipeline

🎯 Module 2: Advanced Text Preprocessing

Now we'll implement a comprehensive text preprocessing pipeline that cleans and normalizes our news articles. This is crucial for all downstream NLP tasks.

Key Preprocessing Steps:

1. **Text Cleaning:** Remove HTML, URLs, special characters
2. **Tokenization:** Split text into individual words
3. **Normalization:** Convert to lowercase, handle contractions
4. **Stop Word Removal:** Remove common words that don't carry meaning
5. **Lemmatization:** Reduce words to their base form

💡 **Think About:** Why is preprocessing so important? What happens if we skip these steps?

✦ Gemini

```
import nltk
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
nltk.download('omw-1.4')

from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
import re
import pandas as pd
from nltk.tokenize import word_tokenize
from collections import Counter

# Initialize preprocessing tools
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    """
    Comprehensive text cleaning function

    💡 TIP: This function should handle:
    - HTML tags and entities
    - URLs and email addresses
    - Special characters and numbers
    - Extra whitespace
    Comprehensive text cleaning function.
    Removes HTML, URLs, emails, special characters
    """
    if pd.isna(text):
        return ""

    # Convert to string and lowercase
    text = str(text).lower()

    # 🚀 YOUR CODE HERE: Implement text cleaning
    # Remove HTML tags
```

```
    text = re.sub(r'<[^>]+>', ' ', text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '',
    text = re.sub(r'http\S+|www\S+|https\S+', '',

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)

    # Remove special characters and digits (keep c
    # Remove special characters and digits
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    return text

def preprocess_text(text, remove_stopwords=True, l
    """
    Complete preprocessing pipeline

    💡 TIP: This function should:
    - Clean the text
    - Tokenize into words
    - Remove stop words (optional)
    - Lemmatize words (optional)
    - Return processed text
    """
    # Clean text
    Complete preprocessing pipeline:
    cleaning, tokenization, stop word removal, lem
    """
    text = clean_text(text)

    if not text:
        return ""

    # 🚀 YOUR CODE HERE: Implement tokenization a
    # Tokenize
    tokens = word_tokenize(text)
    # Simple tokenization after cleaning
    tokens = text.split()

    # Remove stop words if requested
    # Remove stop words
    if remove_stopwords:
        tokens = [token for token in tokens if tok

    # Lemmatize if requested
    # Lemmatize
    if lemmatize:
        tokens = [lemmatizer.lemmatize(token) for

    # Filter out very short words
    # Remove very short words
    tokens = [token for token in tokens if len(tok

    return ' '.join(tokens)
```

```
# Test the preprocessing function
sample_text = "Apple Inc. announced record quarter

print("Original text:")
print(sample_text)

print("\nCleaned text:")
print(clean_text(sample_text))

print("\nFully preprocessed text:")
print(preprocess_text(sample_text))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
Original text:
Apple Inc. announced record quarterly earnings today! Visit https://apple.com for more info.

Cleaned text:
apple inc announced record quarterly earnings today visit for more info technews

Fully preprocessed text:
apple inc announced record quarterly earnings today visit info technews
[nltk_data]   Package omw-1.4 is already up-to-date!
```

```
# Apply preprocessing to the dataset
print("🧹 Preprocessing all articles...")

# Make sure required columns exist
if 'title' not in df.columns:
    df['title'] = df['content'].str.split().str[:8].str.join(' ')

if 'content' not in df.columns:
    raise ValueError("The dataframe must contain a 'content' column.")

# Fill missing text values just in case
df['title'] = df['title'].fillna("")
df['content'] = df['content'].fillna("")

# Create new columns for processed text
df['title_clean'] = df['title'].apply(clean_text)
df['content_clean'] = df['content'].apply(clean_text)
df['title_processed'] = df['title'].apply(preprocess_text)
df['content_processed'] = df['content'].apply(preprocess_text)

# Combine title and content for full article analysis
df['full_text'] = df['title'].astype(str) + ' ' + df['content'].astype(str)
df['full_text_processed'] = df['full_text'].apply(preprocess_text)

print("✅ Preprocessing complete!")

# Show before and after examples
print("\n📝 BEFORE AND AFTER EXAMPLES")
print("=" * 60)
for i in range(min(3, len(df))):
```

```
    print(f"\nExample {i+1}:")
    print(f"Original: {df.iloc[i]['full_text'][:150]}...")
    print(f"Processed: {df.iloc[i]['full_text_processed'][:150]}...")

# Analyze preprocessing results
print("\n📊 PREPROCESSING ANALYSIS")
print("=" * 60)

df['original_word_count'] = df['full_text'].str.split().str.len()
df['processed_word_count'] = df['full_text_processed'].str.split().str.len()

print(f"Average original word count: {df['original_word_count'].mean():.2f}")
print(f"Average processed word count: {df['processed_word_count'].mean():.2f}")

original_words = ' '.join(df['full_text'].astype(str).str.lower()).split()
processed_words = ' '.join(df['full_text_processed'].astype(str)).split()

print(f"Unique words before preprocessing: {len(set(original_words))}")
print(f"Unique words after preprocessing: {len(set(processed_words))}")

common_words = Counter(processed_words).most_common(20)

print("\nMost common words after preprocessing:")
for word, count in common_words:
    print(f"{word}: {count}")
```

```
🧹 Preprocessing all articles...
✅ Preprocessing complete!

📝 BEFORE AND AFTER EXAMPLES
============================================================

Example 1:
Original: worldcom...ex-boss...launches...defence...lawyers...defending...former...worldcom w
Processed: worldcomexbosslaunchesdefencelawyersdefendingformerworldcom worldcom exboss launch

Example 2:
Original: german...business...confidence...slides...german...business...confidence...fell ger
Processed: germanbusinessconfidenceslidesgermanbusinessconfidencefell german business confide

Example 3:
Original: bbc...poll...indicates...economic...gloom...citizens...in...a bbc poll indicates ec
Processed: bbcpollindicateseconomicgloomcitizensina bbc poll indicates economic gloom citizen

📊 PREPROCESSING ANALYSIS
============================================================
Average original word count: 386.01
Average processed word count: 208.08
Unique words before preprocessing: 37025
Unique words after preprocessing: 23919

Most common words after preprocessing:
said: 4838
year: 1872
would: 1711
also: 1426
new: 1334
people: 1323
one: 1190
could: 1032
game: 949
```

```
time: 940
first: 893
last: 883
say: 844
two: 816
world: 811
film: 802
government: 771
make: 695
company: 682
firm: 675
```

📊 Feature Extraction and Statistical Analysis

🎯 Module 3: TF-IDF Analysis

Now we'll extract numerical features from our text using TF-IDF (Term Frequency-Inverse Document Frequency). This technique helps us identify the most important words in each document and across the entire corpus.

TF-IDF Key Concepts:

- **Term Frequency (TF):** How often a word appears in a document
- **Inverse Document Frequency (IDF):** How rare a word is across all documents
- **TF-IDF Score:** TF × IDF - balances frequency with uniqueness

💡 **Business Value:** TF-IDF helps us identify the most distinctive and important terms for each news category.

```
# Create TF-IDF vectorizer
# 💡 TIP: Experiment with different parameters:
# - max_features: limit vocabulary size
# - ngram_range: include phrases (1,1) for words, (1,2) for words+bigrams
# - min_df: ignore terms that appear in less than min_df documents
# - max_df: ignore terms that appear in more than max_df fraction of documents
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,  # Limit vocabulary for computational efficiency
    ngram_range=(1, 2),  # Include unigrams and bigrams
    min_df=2,  # Ignore terms that appear in less than 2 documents
    max_df=0.8  # Ignore terms that appear in more than 80% of documents
)

# Fit and transform the processed text
print("🔢 Creating TF-IDF features...")
tfidf_matrix = tfidf_vectorizer.fit_transform(df['full_text_processed'])
feature_names = tfidf_vectorizer.get_feature_names_out()

print(f"✅ TF-IDF matrix created!")
print(f"📊 Shape: {tfidf_matrix.shape}")
print(f"📝 Vocabulary size: {len(feature_names)}")
print(f"🔢 Sparsity: {(1 - tfidf_matrix.nnz / (tfidf_matrix.shape[0] * tfidf_matrix.shape[1

# Convert to DataFrame for easier analysis
```

```
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)
tfidf_df['category'] = df['category'].values

print("\n🔍 Sample TF-IDF features:")
print(tfidf_df.iloc[:3, :10])  # Show first 3 rows and 10 features
```

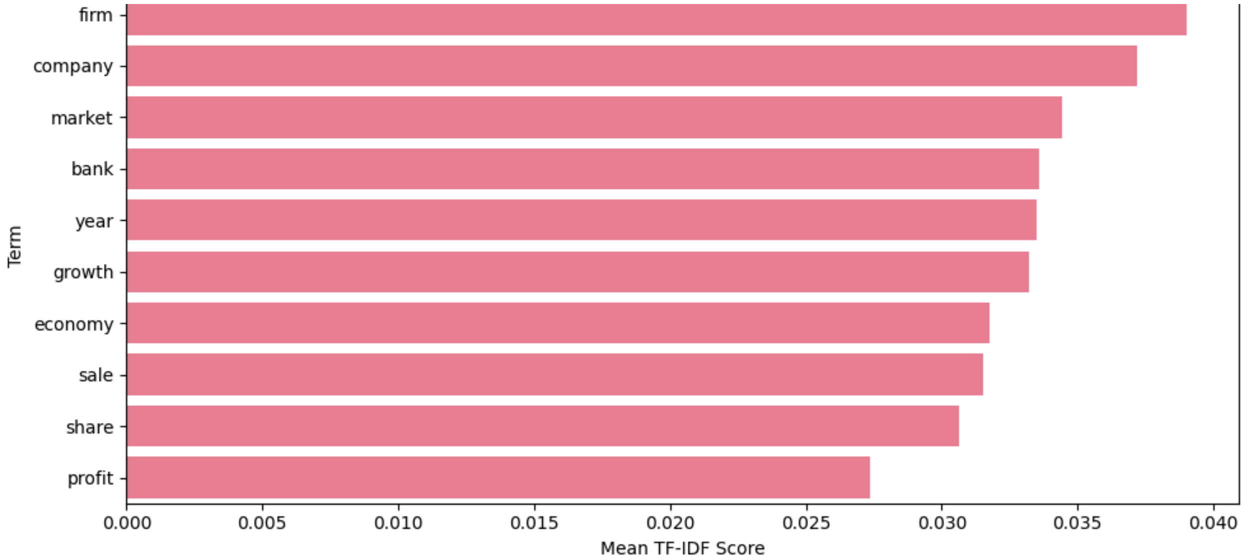
```
🔢 Creating TF-IDF features...
✅ TF-IDF matrix created!
📊 Shape: (1490, 5000)
📝 Vocabulary size: 5000
🔢 Sparsity: 97.50%

🔍 Sample TF-IDF features:
   abc  ability  able  abroad  absence  absolute  absolutely  abuse  abused  \
0  0.0      0.0   0.0     0.0      0.0       0.0         0.0    0.0     0.0
1  0.0      0.0   0.0     0.0      0.0       0.0         0.0    0.0     0.0
2  0.0      0.0   0.0     0.0      0.0       0.0         0.0    0.0     0.0

   academy
0      0.0
1      0.0
2      0.0
```

```
# Analyze most important terms per category
def get_top_tfidf_terms(category, n_terms=10):
    """
    Get top TF-IDF terms for a specific category.
    """
    category_data = tfidf_df[tfidf_df['category'] == category]
    mean_scores = category_data.drop('category', axis=1).mean().sort_values(ascending=False)
    return mean_scores.head(n_terms)

# Analyze top terms for each category
print("🏷️ TOP TF-IDF TERMS BY CATEGORY")
print("=" * 50)

categories = df['category'].unique()
category_terms = {}

for category in categories:
    top_terms = get_top_tfidf_terms(category, n_terms=10)
    category_terms[category] = top_terms

    print(f"\n📰 {category.upper()}:")
    for term, score in top_terms.items():
        print(f"  {term}: {score:.4f}")

# Bar charts of top terms by category
for category, terms in category_terms.items():
    plt.figure(figsize=(10, 5))
    sns.barplot(x=terms.values, y=terms.index)
    plt.title(f"Top TF-IDF Terms for {category}")
    plt.xlabel("Mean TF-IDF Score")
    plt.ylabel("Term")
    plt.tight_layout()
    plt.show()

# Heatmap of top term importance across categories
top_all_terms = list(set([term for terms in category_terms.values() for term in terms.index]
```

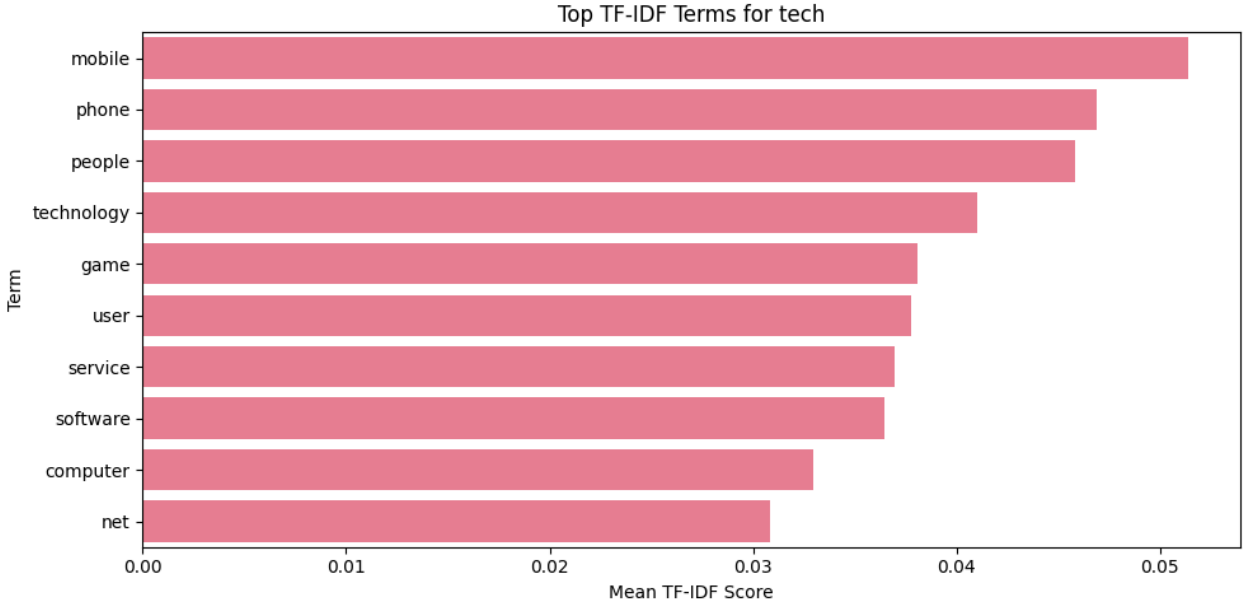
```
heatmap_data = []

for category in categories:
    category_data = tfidf_df[tfidf_df['category'] == category]
    mean_scores = category_data.drop('category', axis=1).mean()
    heatmap_data.append([mean_scores.get(term, 0) for term in top_all_terms])

heatmap_df = pd.DataFrame(heatmap_data, index=categories, columns=top_all_terms)

plt.figure(figsize=(14, 6))
sns.heatmap(heatmap_df, cmap="YlGnBu", annot=False)
plt.title("TF-IDF Term Importance Across Categories")
plt.xlabel("Terms")
plt.ylabel("Category")
plt.tight_layout()
plt.show()
```

```
🏷️ TOP TF-IDF TERMS BY CATEGORY
==================================================

📰 BUSINESS:
  firm: 0.0390
  company: 0.0372
  market: 0.0344
  bank: 0.0336
  year: 0.0335
  growth: 0.0332
  economy: 0.0317
  sale: 0.0316
  share: 0.0307
  profit: 0.0273

📰 TECH:
  mobile: 0.0514
  phone: 0.0469
  people: 0.0458
  technology: 0.0410
  game: 0.0381
  user: 0.0378
  service: 0.0370
  software: 0.0365
  computer: 0.0330
  net: 0.0308

📰 POLITICS:
  labour: 0.0649
  election: 0.0605
  blair: 0.0559
  party: 0.0538
  tory: 0.0462
  would: 0.0456
  government: 0.0455
  minister: 0.0428
  brown: 0.0384
  tax: 0.0329

📰 SPORT:
  game: 0.0448
  england: 0.0374
  win: 0.0341
  player: 0.0322
  match: 0.0305
  champion: 0.0293
  cup: 0.0278
  team: 0.0259
  chelsea: 0.0255
  injury: 0.0249

📰 ENTERTAINMENT:
  film: 0.0995
  award: 0.0524
  best: 0.0460
  show: 0.0376
  star: 0.0372
  music: 0.0356
  band: 0.0349
  actor: 0.0335
  year: 0.0300
  album: 0.0290
```

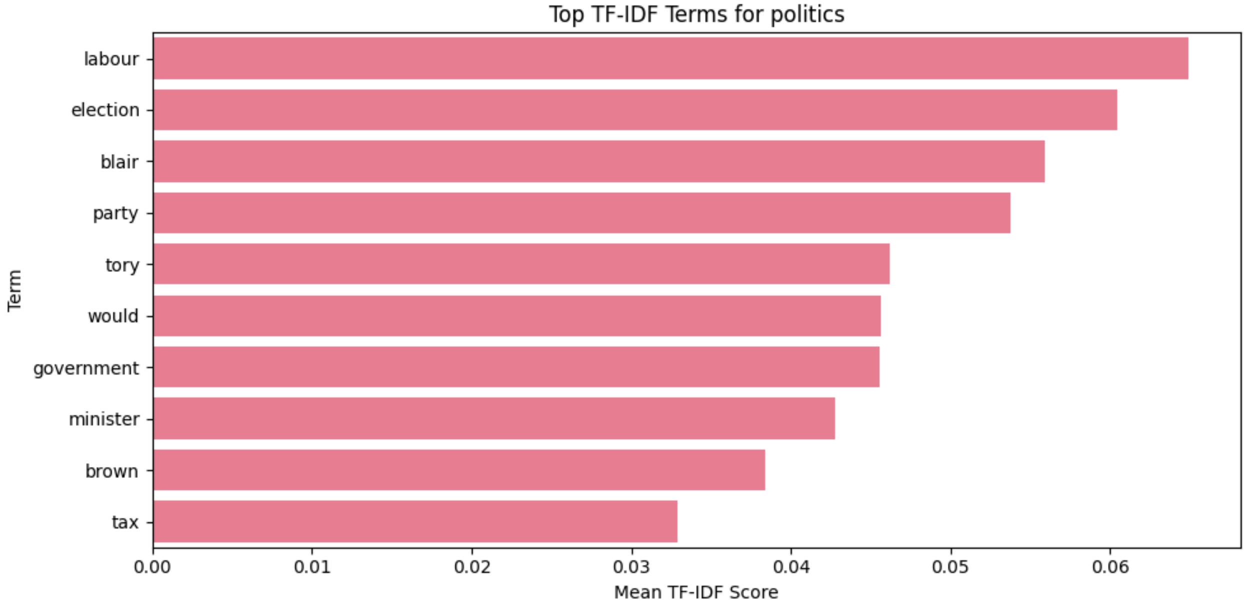
Top TF-IDF Terms for business



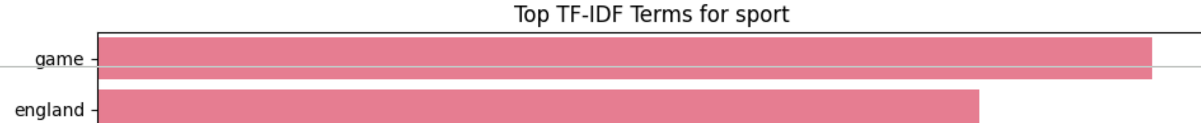
Top TF-IDF Terms for tech

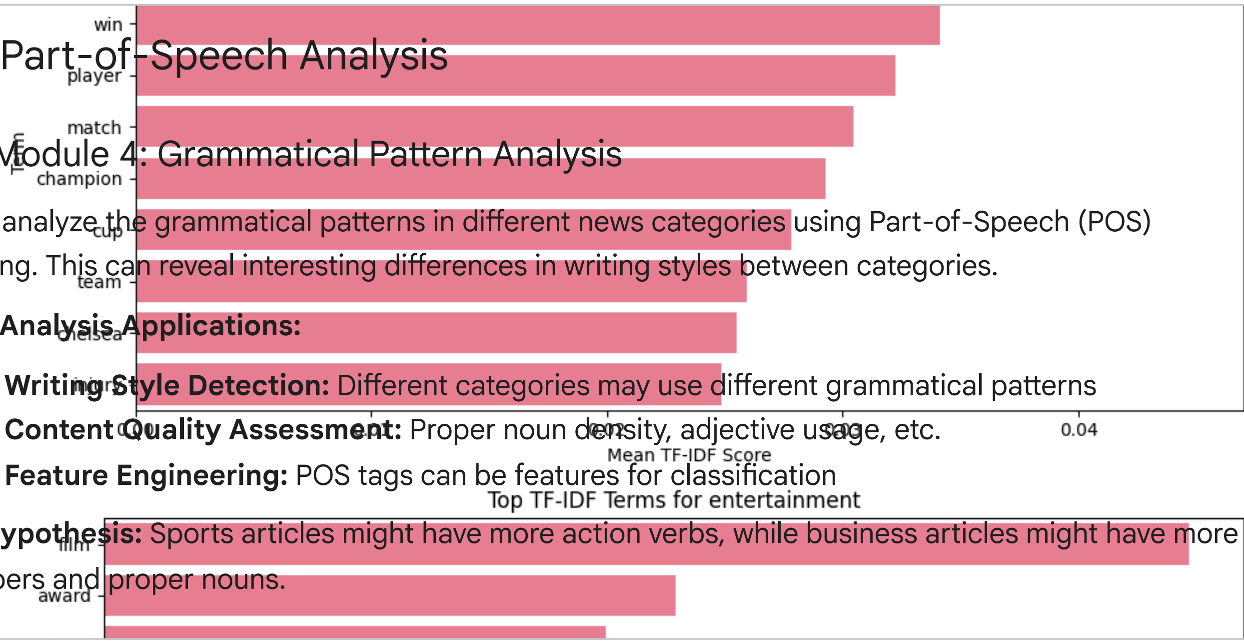


Top TF-IDF Terms for politics



Top TF-IDF Terms for sport





🏷️ Part-of-Speech Analysis

🎯 Module 4: Grammatical Pattern Analysis

Let's analyze the grammatical patterns in different news categories using Part-of-Speech (POS) tagging. This can reveal interesting differences in writing styles between categories.

POS Analysis Applications:

- **Writing Style Detection:** Different categories may use different grammatical patterns
- **Content Quality Assessment:** Proper noun density, adjective usage, etc.
- **Feature Engineering:** POS tags can be features for classification

💡 **Hypothesis:** Sports articles might have more action verbs, while business articles might have more numbers and proper nouns.

```
import nltk
nltk.download('averaged_perceptron_tagger_eng')

from nltk import pos_tag
from collections import Counter

def analyze_pos_patterns(text):
    """
    Analyze POS patterns in text.
    Uses simple split tokenization to avoid NLTK punkt issues.
    """
    if not text or pd.isna(text):
        return {}

    # Clean and tokenize text
    cleaned = clean_text(text)
    tokens = cleaned.split()

    if len(tokens) == 0:
        return {}

    # POS tagging
    pos_tags = pos_tag(tokens)

    # Count POS categories
    pos_counts = Counter([tag for word, tag in pos_tags])
    total_words = len(pos_tags)

    # Convert counts to proportions
    pos_proportions = {pos: count / total_words for pos, count in pos_counts.items()}

    return pos_proportions

# Apply POS analysis to all articles
print("🏷️ Analyzing POS patterns...")

pos_results = []

for idx, row in df.iterrows():
```

```
    pos_analysis = analyze_pos_patterns(row['full_text'])
    pos_analysis['category'] = row['category']
    pos_analysis['article_id'] = row['article_id']
    pos_results.append(pos_analysis)

# Convert to DataFrame
pos_df = pd.DataFrame(pos_results).fillna(0)

print("✅ POS analysis complete!")
print(f"📊 Found {len(pos_df.columns) - 2} different POS tags")

# Show sample results
print("\n📝 Sample POS analysis:")
display(pos_df.head())
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
🏷️ Analyzing POS patterns...
✅ POS analysis complete!
📊 Found 37 different POS tags

📝 Sample POS analysis:
```

	NN	VBZ	NNS	VBG	JJ	IN	DT	VBP	VBN	P
0	0.257627	0.023729	0.122034	0.030508	0.091525	0.118644	0.071186	0.023729	0.044068	0.013
1	0.243671	0.015823	0.075949	0.034810	0.126582	0.145570	0.088608	0.015823	0.031646	0.006
2	0.202454	0.030675	0.089980	0.030675	0.083845	0.137014	0.106339	0.020450	0.034765	0.018
3	0.150641	0.022436	0.115385	0.025641	0.089744	0.131410	0.067308	0.049679	0.022436	0.014
4	0.248571	0.025714	0.088571	0.040000	0.097143	0.134286	0.111429	0.017143	0.028571	0.017

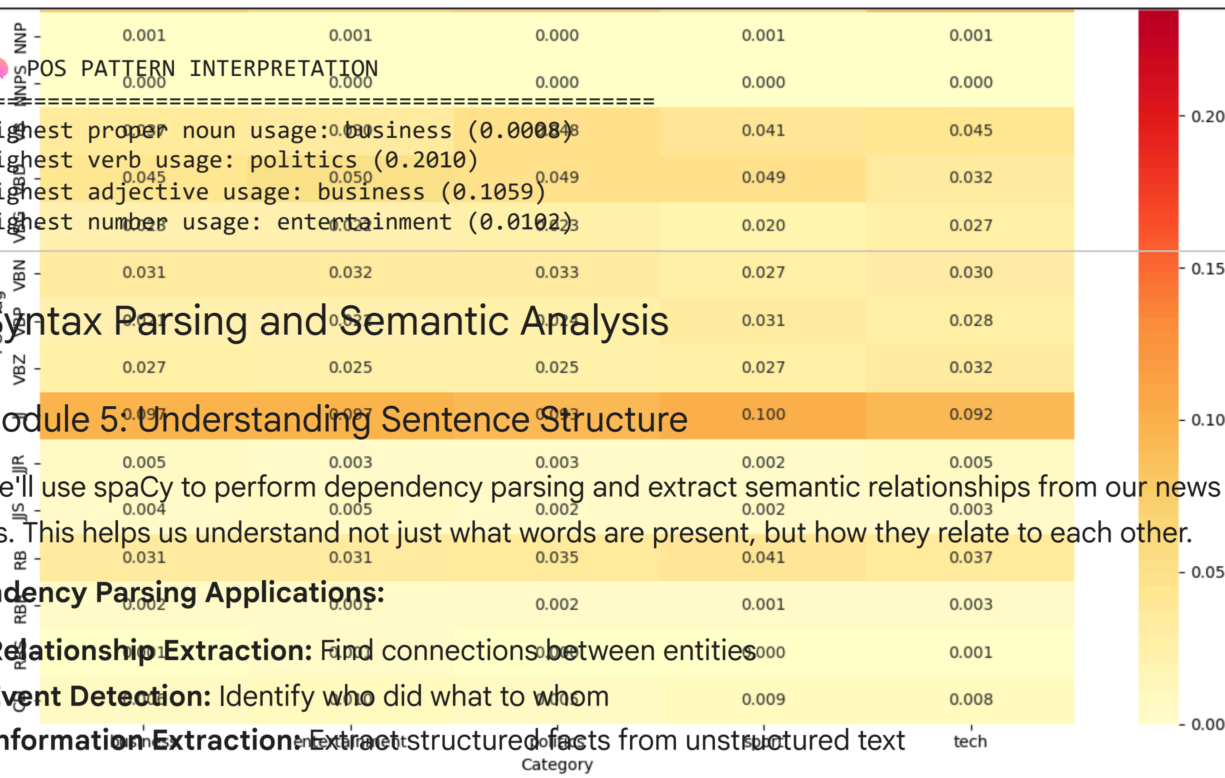
5 rows × 39 columns

```
# Analyze POS patterns by category
print("📊 POS PATTERNS BY CATEGORY")
print("=" * 50)

# Group by category and calculate mean proportions
pos_by_category = pos_df.groupby('category').mean()

# Focus on major POS categories
major_pos = ['NN', 'NNS', 'NNP', 'NNPS', 'VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ',
             'JJ', 'JJR', 'JJS', 'RB', 'RBR', 'RBS', 'CD']

# Filter to only include major POS tags that exist in our data
available_pos = [pos for pos in major_pos if pos in pos_by_category.columns]

if available_pos:
    pos_summary = pos_by_category[available_pos]

    print("\n🎯 Key POS patterns by category:")
    print(pos_summary.round(4))

    # Create visualization
```

```
    plt.figure(figsize=(12, 8))
    sns.heatmap(pos_summary.T, annot=True, cmap='YlOrRd', fmt='.3f')
    plt.title('POS Tag Proportions by News Category')
    plt.xlabel('Category')
    plt.ylabel('POS Tag')
    plt.tight_layout()
    plt.show()

    # 💡 STUDENT TASK: Analyze the patterns
    # - Which categories use more nouns vs verbs?
    # - Do business articles have more numbers (CD)?
    # - Are there differences in adjective usage?

    print("\n💡 ANALYSIS QUESTIONS:")
    print("1. Which category has the highest proportion of proper nouns (NNP/NNPS)?")
    print("2. Which category uses the most action verbs (VB, VBD, VBG)?")
    print("3. Are there interesting patterns in adjective (JJ) usage?")
    print("4. How does number (CD) usage vary across categories?")
else:
    print("⚠️ No major POS tags found in the analysis. Check your POS tagging implementatio
```

📊 POS PATTERNS BY CATEGORY

```
# Additional POS interpretation for student task

print("\n🧠 POS PATTERN INTERPRETATION")
print("=" * 50)

# Proper nouns
proper_noun_cols = [col for col in ['NNP', 'NNPS'] if col in pos_by_category.columns]
if proper_noun_cols:
    proper_nouns = pos_by_category[proper_noun_cols].sum(axis=1)
    print(f"Highest proper noun usage: {proper_nouns.idxmax()} ({proper_nouns.max():.4f})")

# Action verbs
verb_cols = [col for col in ['VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ'] if col in pos_by_catego
if verb_cols:
    verbs = pos_by_category[verb_cols].sum(axis=1)
    print(f"Highest verb usage: {verbs.idxmax()} ({verbs.max():.4f})")

# Adjectives
adj_cols = [col for col in ['JJ', 'JJR', 'JJS'] if col in pos_by_category.columns]
if adj_cols:
    adjectives = pos_by_category[adj_cols].sum(axis=1)
    print(f"Highest adjective usage: {adjectives.idxmax()} ({adjectives.max():.4f})")

# Numbers
if 'CD' in pos_by_category.columns:
    numbers = pos_by_category['CD']
    print(f"Highest number usage: {numbers.idxmax()} ({numbers.max():.4f})")
```




```
🧠 POS PATTERN INTERPRETATION
==================================================
Highest proper noun usage: business (0.0008)
Highest verb usage: politics (0.2010)
Highest adjective usage: business (0.1059)
Highest number usage: entertainment (0.0102)
```

🌳 Syntax Parsing and Semantic Analysis

🎯 Module 5: Understanding Sentence Structure

Now we'll use spaCy to perform dependency parsing and extract semantic relationships from our news articles. This helps us understand not just what words are present, but how they relate to each other.

Dependency Parsing Applications:

- **Relationship Extraction:** Find connections between entities
- **Event Detection:** Identify who did what to whom
- **Information Extraction:** Extract structured facts from unstructured text

💡 **Business Value:** Understanding sentence structure helps extract more precise information about events, relationships, and actions mentioned in news articles.

```
💡 ANALYSIS QUESTIONS:
1. Which category has the highest proportion of proper nouns (NNP/NNPS)?
2. Which category uses the most action verbs (VB, VBD, VBG)?
```

```
# Install/load spaCy English model if needed
import spacy
from collections import Counter
```

```
from collections import Counter
import pandas as pd

try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    !python -m spacy download en_core_web_sm
    nlp = spacy.load("en_core_web_sm")

def extract_syntactic_features(text):
    """
    Extract syntactic features using spaCy dependency parsing.
    """
    if not text or pd.isna(text):
        return {}

    # Limit length for speed in Colab
    doc = nlp(str(text)[:3000])

    features = {
        'num_sentences': len(list(doc.sents)),
        'num_tokens': len(doc),
        'dependency_relations': [],
        'noun_phrases': [],
        'verb_phrases': [],
        'subjects': [],
        'objects': []
    }

    # Extract dependency relations
    for token in doc:
        if not token.is_space and not token.is_punct:
            features['dependency_relations'].append(token.dep_)

    # Extract noun phrases
    for chunk in doc.noun_chunks:
        features['noun_phrases'].append(chunk.text.lower())

    # Extract simple verb phrases
    for token in doc:
        if token.pos_ == "VERB":
            phrase = " ".join([child.text for child in token.children if child.dep_ in ["aux",
            features['verb_phrases'].append(phrase.lower())

    # Extract subjects and objects
    for token in doc:
        if token.dep_ in ['nsubj', 'nsubjpass']:
            features['subjects'].append(token.text.lower())
        elif token.dep_ in ['dobj', 'obj', 'iobj', 'pobj']:
            features['objects'].append(token.text.lower())

    # Count dependency types
    dep_counts = Counter(features['dependency_relations'])
    features['dependency_counts'] = dict(dep_counts)

    return features

# Apply syntactic analysis to sample articles
print("🌳 Performing syntactic analysis...")
```

```
syntactic_results = []

for idx, row in df.head(5).iterrows():
    features = extract_syntactic_features(row['full_text'])
    features['category'] = row['category']
    features['article_id'] = row['article_id']
    syntactic_results.append(features)

print("✅ Syntactic analysis complete!")

# Display results
for i, result in enumerate(syntactic_results):
    print(f"\n📰 Article {i+1} ({result['category']}):")
    print(f"  Sentences: {result['num_sentences']}")
    print(f"  Tokens: {result['num_tokens']}")
    print(f"  Noun phrases: {result['noun_phrases'][:3]}...")
    print(f"  Verb phrases: {result['verb_phrases'][:3]}...")
    print(f"  Subjects: {result['subjects'][:3]}...")
    print(f"  Objects: {result['objects'][:3]}...")
```

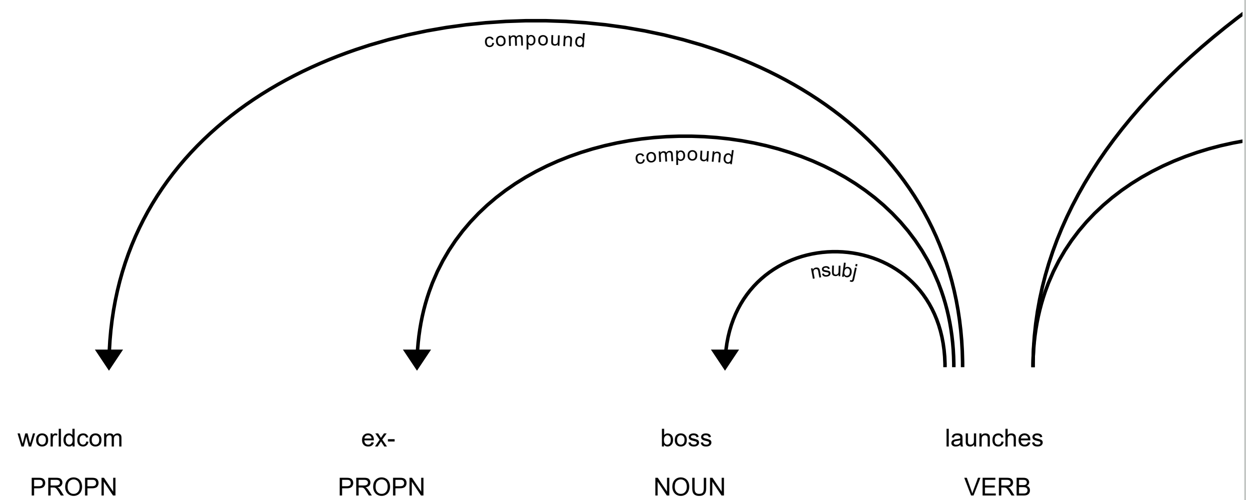
```
🌳 Performing syntactic analysis...
✅ Syntactic analysis complete!

📰 Article 1 (business):
  Sentences: 15
  Tokens: 363
  Noun phrases: ['worldcom', 'ex-boss...launches', 'defence']...
  Verb phrases: ['defending', 'launches', 'defending']...
  Subjects: ['worldcom', '-', 'boss']...
  Objects: ['ebbers', 'battery', 'charges']...

📰 Article 2 (business):
  Sentences: 15
  Tokens: 383
  Noun phrases: ['german', 'business', 'confidence']...
  Verb phrases: ['fell', 'slides', 'fell']...
  Subjects: ['german', 'confidence', 'ifo']...
  Objects: ['confidence', 'february', 'hopes']...

📰 Article 3 (business):
  Sentences: 22
  Tokens: 569
  Noun phrases: ['bbc...poll', 'economic...gloom', 'citizens']...
  Verb phrases: ['indicates', 'indicates', 'surveyed']...
  Subjects: ['poll', 'gloom', 'poll']...
  Objects: ['citizens', 'majority', 'nations']...

📰 Article 4 (tech):
  Sentences: 24
  Tokens: 604
  Noun phrases: ['lifestyle...governs', 'mobile...choice', '...funkier lifestyle  governs']..
  Verb phrases: ['is not going', 'to help', 'sell']...
  Subjects: ['governs', 'firms', 'firms']...
  Objects: ['choice', 'hardware', 'research']...

📰 Article 5 (business):
  Sentences: 17
  Tokens: 432
  Noun phrases: ['enron', 'bosses', '$168m']...
  Verb phrases: ['m', 'have agreed', 'leading']...
```

```
  Subjects: ['enron', 'plaintiff', 'news']...
  Objects: ['$168m', 'payout', '168']...
```

```
# Visualize dependency parsing for a sample sentence
from spacy import displacy

# Choose a sample sentence
sample_sentence = df.iloc[0]['content']  # First article's content
print(f"📝 Sample sentence: {sample_sentence}")

# Process with spaCy
doc = nlp(sample_sentence)

# Display dependency tree (this works best in Jupyter)
print("\n🌳 Dependency Parse Visualization:")
try:
    # This will create an interactive visualization in Jupyter
    displacy.render(doc, style="dep", jupyter=True)
except:
    # Fallback: print dependency information
    print("\n🔗 Dependency Relations:")
    for token in doc:
        if not token.is_space and not token.is_punct:
            print(f"  {token.text} --> {token.dep_} --> {token.head.text}")

# 💡 STUDENT TASK: Extend syntactic analysis
# - Compare syntactic complexity across categories
# - Extract action patterns (who did what)
# - Identify most common dependency relations per category
# - Create features for classification based on syntax
```

📝 Sample sentence: worldcom ex-boss launches defence lawyers defending former worldcom chie

🌳 Dependency Parse Visualization:




```
print("🌳 SYNTACTIC COMPLEXITY BY CATEGORY")
print("=" * 60)

syntax_summary = []

# Analyze a sample from each category for speed
for category in df['category'].unique():
    category_sample = df[df['category'] == category].head(25)

    sentence_counts = []
    token_counts = []
    noun_phrase_counts = []
    dependency_counter = Counter()
```

```
    action_patterns = []

    for _, row in category_sample.iterrows():
        doc = nlp(str(row['full_text'])[:3000])

        sentence_counts.append(len(list(doc.sents)))
        token_counts.append(len(doc))
        noun_phrase_counts.append(len(list(doc.noun_chunks)))

        for token in doc:
            if not token.is_space and not token.is_punct:
                dependency_counter[token.dep_] += 1

            # Simple action pattern: subject → verb → object
            if token.pos_ == "VERB":
                subjects = [child.text for child in token.children if child.dep_ in ["nsubj",
                objects = [child.text for child in token.children if child.dep_ in ["dobj", "

                for subj in subjects:
                    for obj in objects:
                        action_patterns.append(f"{subj} → {token.text} → {obj}")

    syntax_summary.append({
        "category": category,
        "avg_sentences": sum(sentence_counts) / len(sentence_counts),
        "avg_tokens": sum(token_counts) / len(token_counts),
        "avg_noun_phrases": sum(noun_phrase_counts) / len(noun_phrase_counts),
        "most_common_dependencies": dependency_counter.most_common(5),
        "sample_action_patterns": action_patterns[:5]
    })

syntax_summary_df = pd.DataFrame(syntax_summary)

display(syntax_summary_df[["category", "avg_sentences", "avg_tokens", "avg_noun_phrases"]])

print("\n🔗 MOST COMMON DEPENDENCY RELATIONS BY CATEGORY")
print("=" * 60)

for row in syntax_summary:
    print(f"\n📰 {row['category'].upper()}:")
    print(row["most_common_dependencies"])

print("\n⚡ SAMPLE ACTION PATTERNS")
print("=" * 60)

for row in syntax_summary:
    print(f"\n📰 {row['category'].upper()}:")
    if row["sample_action_patterns"]:
        for pattern in row["sample_action_patterns"]:
            print(f"  {pattern}")
    else:
        print("  No clear subject-verb-object patterns found in sample.")
```

```
🌳 SYNTACTIC COMPLEXITY BY CATEGORY
============================================================
```

	category	avg_sentences	avg_tokens	avg_noun_phrases
0	business	14.04	361.44	82.00
1	tech	20.60	503.56	118.12
2	politics	17.84	465.28	107.76
3	sport	16.92	358.24	82.36
4	entertainment	14.52	367.92	85.56

```
🔗 MOST COMMON DEPENDENCY RELATIONS BY CATEGORY
============================================================

📰 BUSINESS:
[('prep', 905), ('pobj', 865), ('compound', 691), ('nsubj', 645), ('det', 635)]

📰 TECH:
[('prep', 1111), ('pobj', 1059), ('det', 998), ('nsubj', 858), ('compound', 856)]

📰 POLITICS:
[('prep', 959), ('nsubj', 958), ('det', 942), ('pobj', 930), ('compound', 843)]
```

😊 Sentiment and Emotion Analysis

🎯 Module 6: Understanding Emotional Tone

Let's analyze the sentiment and emotional tone of our news articles. This can reveal interesting patterns about how different types of news are presented and perceived.

Sentiment Analysis Applications:

- **Media Bias Detection:** Identify emotional slant in news coverage
- **Public Opinion Tracking:** Monitor sentiment trends over time
- **Content Recommendation:** Suggest articles based on emotional tone

💡 **Hypothesis:** Different news categories might have different emotional profiles - sports might be more positive, politics more negative, etc.

```python
import nltk
nltk.download('vader_lexicon')

from nltk.sentiment import SentimentIntensityAnalyzer
import pandas as pd

# Initialize sentiment analyzer
sia = SentimentIntensityAnalyzer()

def analyze_sentiment(text):
    """
    Analyze sentiment using VADER sentiment analyzer.
    """
    if not text or pd.isna(text):
        return {'compound': 0, 'pos': 0, 'neu': 1, 'neg': 0, 'sentiment_label': 'neutral'}

    scores = sia.polarity_scores(str(text))

    if scores['compound'] >= 0.05:
        scores['sentiment_label'] = 'positive'
    elif scores['compound'] <= -0.05:
        scores['sentiment_label'] = 'negative'
    else:
        scores['sentiment_label'] = 'neutral'

    return scores

# Apply sentiment analysis to all articles
print("😊 Analyzing sentiment...")

# Make sure full_text exists
if 'full_text' not in df.columns:
    df['full_text'] = df['title'].astype(str) + ' ' + df['content'].astype(str)
```

```
sentiment_results = []

for idx, row in df.iterrows():
    title_sentiment = analyze_sentiment(row['title'])
    content_sentiment = analyze_sentiment(row['content'])
    full_sentiment = analyze_sentiment(row['full_text'])

    result = {
        'article_id': row['article_id'],
        'category': row['category'],
        'title_sentiment': title_sentiment['compound'],
        'title_label': title_sentiment['sentiment_label'],
        'content_sentiment': content_sentiment['compound'],
        'content_label': content_sentiment['sentiment_label'],
        'full_sentiment': full_sentiment['compound'],
        'full_label': full_sentiment['sentiment_label'],
        'pos_score': full_sentiment['pos'],
        'neu_score': full_sentiment['neu'],
        'neg_score': full_sentiment['neg']
    }

    sentiment_results.append(result)

# Convert to DataFrame
sentiment_df = pd.DataFrame(sentiment_results)

print("✅ Sentiment analysis complete!")
print(f"📊 Analyzed {len(sentiment_df)} articles")

# Display sample results
print("\n📝 Sample sentiment results:")
display(sentiment_df[['category', 'full_sentiment', 'full_label']].head())
```

```
😊 Analyzing sentiment...
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
✅ Sentiment analysis complete!
📊 Analyzed 1490 articles

📝 Sample sentiment results:
```

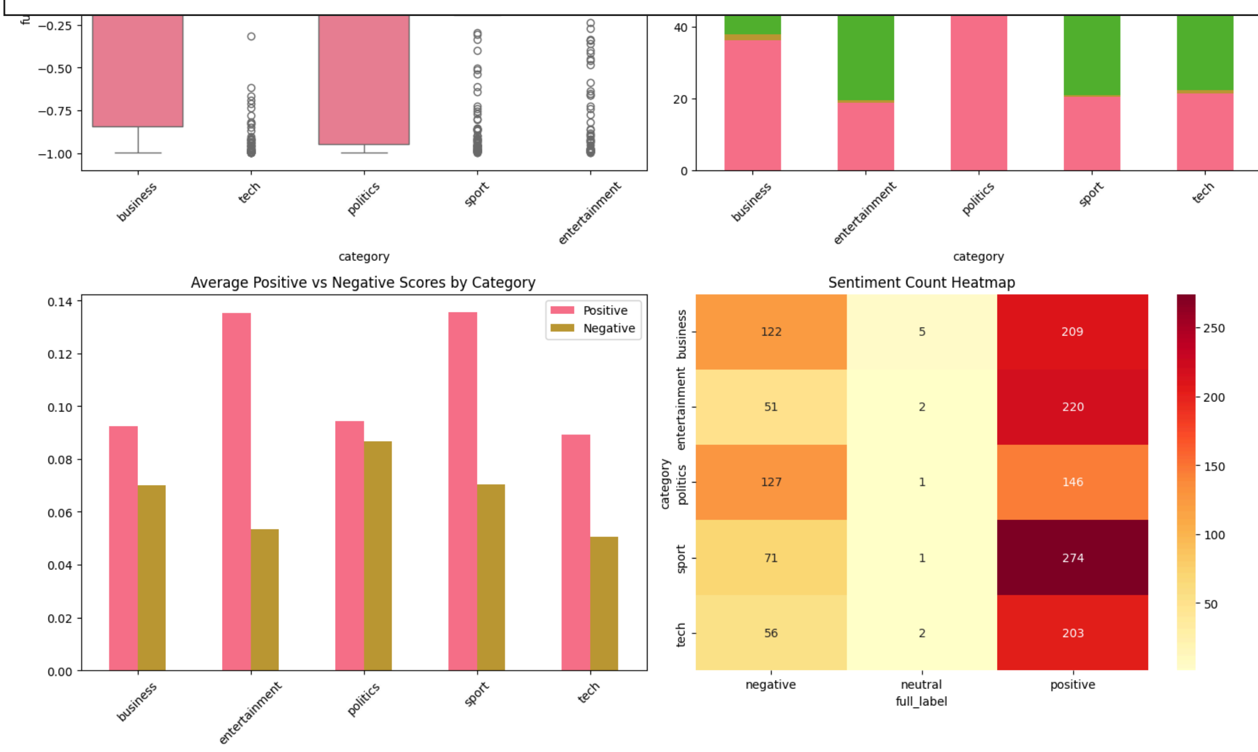
	category	full_sentiment	full_label
0	business	-0.9701	negative
1	business	0.7623	positive
2	business	-0.9318	negative
3	tech	0.9554	positive
4	business	-0.9486	negative

```
# Analyze sentiment patterns by category
print("📊 SENTIMENT ANALYSIS BY CATEGORY")
print("=" * 50)

# Calculate sentiment statistics by category
sentiment_by_category = sentiment_df.groupby('category').agg({
    'full_sentiment': ['mean', 'std', 'min', 'max'],
```

```
    'pos_score': 'mean',
    'neu_score': 'mean',
    'neg_score': 'mean'
}).round(4)

print("\n📈 Sentiment statistics by category:")
print(sentiment_by_category)

# Sentiment distribution by category
sentiment_dist = sentiment_df.groupby(['category', 'full_label']).size().unstack(fill_value=
sentiment_dist_pct = sentiment_dist.div(sentiment_dist.sum(axis=1), axis=0) * 100

print("\n📊 Sentiment distribution (%) by category:")
print(sentiment_dist_pct.round(2))

# Create visualizations
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# 1. Sentiment scores by category
sns.boxplot(data=sentiment_df, x='category', y='full_sentiment', ax=axes[0,0])
axes[0,0].set_title('Sentiment Score Distribution by Category')
axes[0,0].tick_params(axis='x', rotation=45)

# 2. Sentiment label distribution
sentiment_dist_pct.plot(kind='bar', ax=axes[0,1], stacked=True)
axes[0,1].set_title('Sentiment Label Distribution by Category (%)')
axes[0,1].tick_params(axis='x', rotation=45)
axes[0,1].legend(title='Sentiment')

# 3. Positive vs Negative scores
category_means = sentiment_df.groupby('category')[['pos_score', 'neg_score']].mean()
category_means.plot(kind='bar', ax=axes[1,0])
axes[1,0].set_title('Average Positive vs Negative Scores by Category')
axes[1,0].tick_params(axis='x', rotation=45)
axes[1,0].legend(['Positive', 'Negative'])

# 4. Sentiment vs Category heatmap
sentiment_pivot = sentiment_df.pivot_table(values='full_sentiment', index='category',
                                          columns='full_label', aggfunc='count', fill_value=0
sns.heatmap(sentiment_pivot, annot=True, fmt='d', ax=axes[1,1], cmap='YlOrRd')
axes[1,1].set_title('Sentiment Count Heatmap')

plt.tight_layout()
plt.show()

# 💡 STUDENT TASK: Analyze sentiment patterns
# - Which categories are most positive/negative?
# - Are there differences between title and content sentiment?
# - How does sentiment vary within categories?
# - Can sentiment be used as a feature for classification?
```

📊 SENTIMENT ANALYSIS BY CATEGORY

```
)

 than TF-IDF because news categories are mostly defined by topic rather than emotional tone."
```



```
🧠 SENTIMENT INTERPRETATION
==================================================
Most positive category: entertainment (0.5991)
Most negative category: politics (0.0578)

Title vs Content Sentiment:
```

category	avg_title_sentiment	avg_content_sentiment	difference
business	0.0	0.2404	0.2404
entertainment	0.0	0.5991	0.5991
politics	0.0	0.0578	0.0578
sport	0.0	0.5541	0.5541
tech	0.0	0.5233	0.5233

```
Sentiment variation within categories:
```

category	full_sentiment
politics	0.8811
business	0.8370
tech	0.7557
sport	0.7244
entertainment	0.6796

dtype: float64

```
Classification Insight:
Sentiment can be used as an additional feature for classification, but it is probably weaker
```

🏷️ Text Classification System

🎯 Module 7: Building the News Classifier

Now we'll build the core of our NewsBot system - a multi-class text classifier that can automatically categorize news articles. We'll compare different algorithms and evaluate their performance.

Classification Pipeline:

1. **Feature Engineering:** Combine TF-IDF with other features
2. **Model Training:** Train multiple algorithms
3. **Model Evaluation:** Compare performance metrics
4. **Model Selection:** Choose the best performing model

💡 **Business Impact:** Accurate classification enables automatic content routing, personalized recommendations, and efficient content management.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Prepare features for classification
print("🔧 Preparing features for classification...")

# TF-IDF features
X_tfidf = tfidf_matrix.toarray()

# Sentiment features
sentiment_features = sentiment_df[
    ['full_sentiment', 'pos_score', 'neu_score', 'neg_score']
].values

# Text length features
length_features = np.array([
    df['full_text'].astype(str).str.len(),
    df['full_text'].astype(str).str.split().str.len(),
    df['title'].astype(str).str.len()
]).T

# Scale numeric features
scaler = StandardScaler()
numeric_features = scaler.fit_transform(
    np.hstack([sentiment_features, length_features])
)

# Combine TF-IDF and numeric features
X_combined = np.hstack([
    X_tfidf,
    numeric_features
])

# Target variable
y = df['category'].values

print("✅ Feature matrix prepared!")
print(f"📊 Feature matrix shape: {X_combined.shape}")
print(f"🎯 Number of classes: {len(np.unique(y))}")
print(f"📋 Classes: {np.unique(y)}")

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_combined,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\n📈 Data split:")
print(f"  Training set: {X_train.shape[0]} samples")
print(f"  Test set: {X_test.shape[0]} samples")
```

```
🔧 Preparing features for classification...
✅ Feature matrix prepared!
📊 Feature matrix shape: (1490, 5007)
🎯 Number of classes: 5
```

```
📋 Classes: ['business' 'entertainment' 'politics' 'sport' 'tech']

📈 Data split:
  Training set: 1192 samples
  Test set: 298 samples
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score
import pandas as pd

# Train and evaluate multiple classifiers
print("🤖 Training multiple classifiers...")

# MultinomialNB cannot use scaled features with negative values,
# so GaussianNB is safer for this combined feature matrix.
classifiers = {
    'Naive Bayes': GaussianNB(),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'SVM': SVC(random_state=42, probability=True)
}

results = {}
trained_models = {}

for name, classifier in classifiers.items():
    print(f"\n🔄 Training {name}...")

    classifier.fit(X_train, y_train)

    y_pred = classifier.predict(X_test)
    y_pred_proba = classifier.predict_proba(X_test) if hasattr(classifier, 'predict_proba') e

    accuracy = accuracy_score(y_test, y_pred)
    cv_scores = cross_val_score(classifier, X_train, y_train, cv=3, scoring='accuracy')

    results[name] = {
        'accuracy': accuracy,
        'cv_mean': cv_scores.mean(),
        'cv_std': cv_scores.std(),
        'predictions': y_pred,
        'probabilities': y_pred_proba
    }

    trained_models[name] = classifier

    print(f"  ✅ Accuracy: {accuracy:.4f}")
    print(f"  📊 CV Score: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

print("\n🏆 CLASSIFIER COMPARISON")
print("=" * 50)

comparison_df = pd.DataFrame({
    'Model': list(results.keys()),
    'Test Accuracy': [results[name]['accuracy'] for name in results.keys()],
    'CV Mean': [results[name]['cv_mean'] for name in results.keys()],
    'CV Std': [results[name]['cv_std'] for name in results.keys()],
```

```
})

display(comparison_df.round(4))

best_model_name = comparison_df.loc[comparison_df['Test Accuracy'].idxmax(), 'Model']
best_model = trained_models[best_model_name]

print(f"\n🥇 Best performing model: {best_model_name}")
```

```
🤖 Training multiple classifiers...

🔄 Training Naive Bayes...
  ✅ Accuracy: 0.9295
  📊 CV Score: 0.9035 (+/- 0.0121)

🔄 Training Logistic Regression...
  ✅ Accuracy: 0.9597
  📊 CV Score: 0.9186 (+/- 0.0600)

🔄 Training SVM...
  ✅ Accuracy: 0.6812
  📊 CV Score: 0.6451 (+/- 0.0706)

🏆 CLASSIFIER COMPARISON
==================================================
```

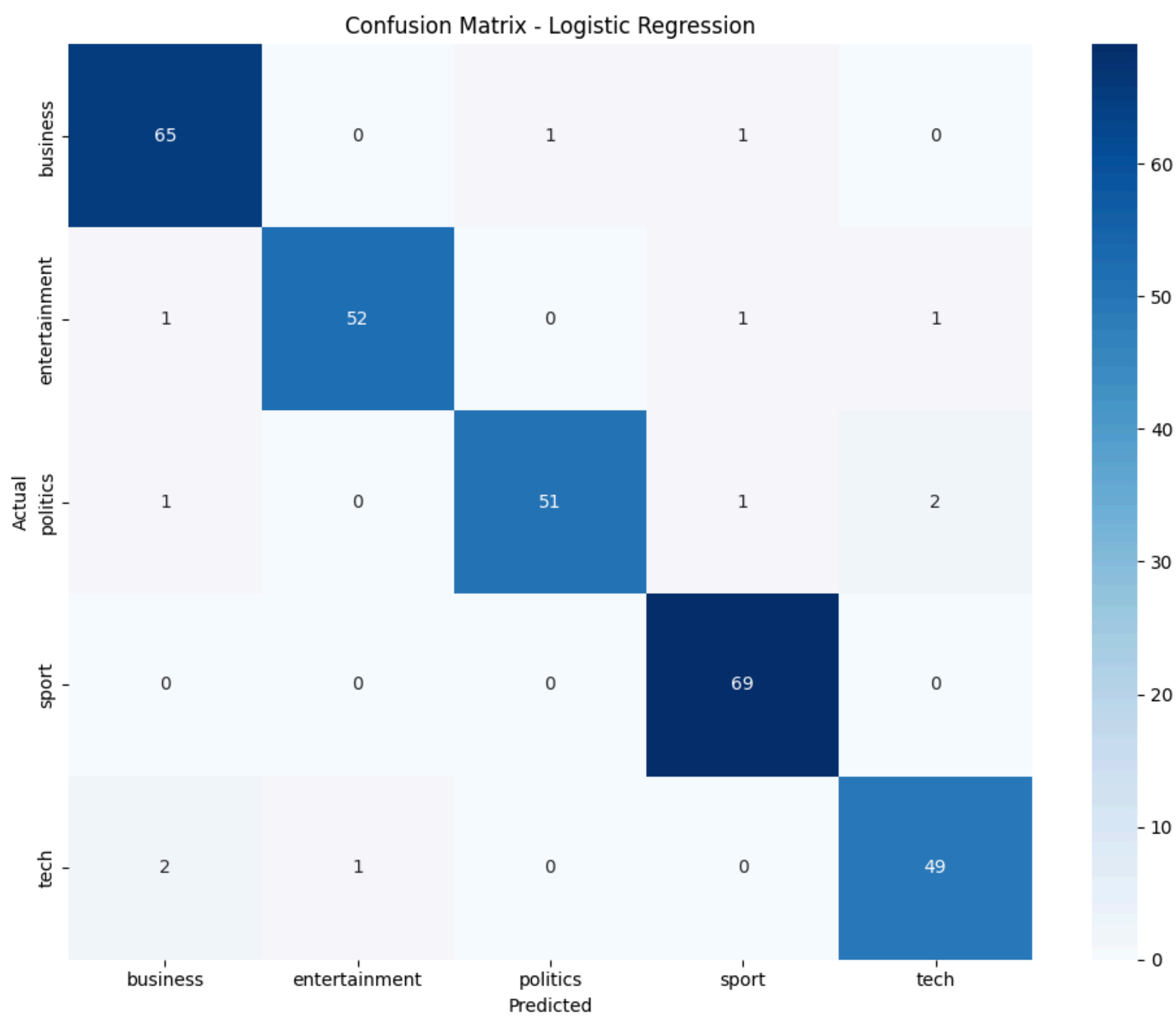
	Model	Test Accuracy	CV Mean	CV Std
0	Naive Bayes	0.9295	0.9035	0.0060
1	Logistic Regression	0.9597	0.9186	0.0300
2	SVM	0.6812	0.6451	0.0353

```
🥇 Best performing model: Logistic Regression
```

```
sklearn.metrics import classification_report, confusion_matrix
t numpy as np

ailed evaluation of the best model
model = trained_models[best_model_name]
predictions = results[best_model_name]['predictions']

(f"📊 DETAILED EVALUATION: {best_model_name}")
("=" * 60)

ssification report
("\n📋 Classification Report:")
(classification_report(y_test, best_predictions))

fusion matrix
s = sorted(np.unique(y))

confusion_matrix(y_test, best_predictions, labels=labels)

igure(figsize=(10, 8))
eatmap(
m,
nnot=True,
mt='d',
```

```
ticklabels=labels,
ticklabels=labels


itle(f'Confusion Matrix - {best_model_name}')
label('Predicted')
label('Actual')
ight_layout()
how()

ture importance for Logistic Regression only
st_model_name == 'Logistic Regression':
rint("\n🔍 Top Features by Category:")

eature_names_extended = (
   list(feature_names) +
   ['full_sentiment', 'pos_score', 'neu_score', 'neg_score',
    'char_length', 'word_count', 'title_length']


lasses = best_model.classes_
oefficients = best_model.coef_

or i, class_name in enumerate(classes):
   top_indices = np.argsort(coefficients[i])[-10:]

   print(f"\n📰 {class_name}:")
   for idx in reversed(top_indices):
       if idx < len(feature_names_extended):
           print(f"  {feature_names_extended[idx]}: {coefficients[i][idx]:.4f}")

dent task: classifier improvement notes
("\n💡 CLASSIFIER IMPROVEMENT NOTES")
("=" * 60)
("1. TF-IDF is the strongest feature type because the news categories are topic-based.")
("2. Sentiment and length features can provide extra context, but they are weaker than word-ba
("3. Hyperparameter tuning could improve Logistic Regression or SVM performance.")
("4. The class distribution is reasonably balanced, so class imbalance is not a major issue."
```

```
📊 DETAILED EVALUATION: Logistic Regression
============================================================

📋 Classification Report:
               precision    recall  f1-score   support

     business       0.94      0.97      0.96        67
entertainment       0.98      0.95      0.96        55
     politics       0.98      0.93      0.95        55
        sport       0.96      1.00      0.98        69
         tech       0.94      0.94      0.94        52

     accuracy                           0.96       298
    macro avg       0.96      0.96      0.96       298
 weighted avg       0.96      0.96      0.96       298
```




```
🔍 Top Features by Category:

📰 business:
  firm: 1.9056
  bank: 1.8096
  company: 1.7605
  market: 1.5074
  economy: 1.4719
  share: 1.4427
  economic: 1.3691
  profit: 1.3094
```

🔍 Named Entity Recognition

```
growth: 1.2850
business: 1.2369
entertainment:
film: 3.2802
show: 2.0295
music: 1.7837
star: 1.6594
album: 1.5731
singer: 1.5338
band: 1.5237
actor: 1.3706
song: 1.2663
chart: 1.2005
politics:
party: 2.1858
labour: 2.0282
blair: 1.9660
```

🎯 Module 8: Extracting Facts from News

Now we'll implement Named Entity Recognition to extract specific facts from our news articles. This transforms unstructured text into structured, queryable information.

NER Applications:

- **Entity Tracking:** Monitor mentions of people, organizations, locations
- **Fact Extraction:** Build knowledge bases from news content
- **Relationship Mapping:** Understand connections between entities
- **Timeline Construction:** Track events and their participants

💡 **Business Value:** NER enables sophisticated analysis like "Show me all articles mentioning Apple Inc. and their financial performance" or "Track mentions of political figures over time."

```
import spacy
import pandas as pd

def extract_entities(text):
    """
    Extract named entities using spaCy.
    """
    if not text or pd.isna(text):
        return []

    # Limit very long articles for speed
    doc = nlp(str(text)[:3000])

    entities = []

    for ent in doc.ents:
        entities.append({
            'text': ent.text,
            'label': ent.label_,
            'start': ent.start_char,
            'end': ent.end_char,
            'description': spacy.explain(ent.label_)
        })

    return entities


# Apply NER to all articles
print("🔍 Extracting named entities...")

all_entities = []
article_entities = []

# You may change 200 to len(df) if you want to process everything.
sample_size = min(200, len(df))

for idx, row in df.head(sample_size).iterrows():

    entities = extract_entities(row['full_text'])

    article_entities.append({
```

```
            'article_id': row['article_id'],
            'category': row['category'],
            'entities': entities,
            'entity_count': len(entities)
        })

        for entity in entities:
            entity['article_id'] = row['article_id']
            entity['category'] = row['category']
            all_entities.append(entity)

    print("✅ Entity extraction complete!")
    print(f"📊 Total entities found: {len(all_entities)}")
    print(f"📰 Articles processed: {sample_size}")

    # Convert to DataFrame
    entities_df = pd.DataFrame(all_entities)

    if not entities_df.empty:
        print(f"\n🏷️ Entity types found:")
        print(entities_df['label'].value_counts())

        print("\n📝 Sample entities:")
        display(
            entities_df[
                ['text', 'label', 'category']
            ].head(10)
        )
    else:
        print("⚠️ No entities found.")
```

```
🔍 Extracting named entities...
✅ Entity extraction complete!
📊 Total entities found: 5440
📰 Articles processed: 200

🏷️ Entity types found:
label
DATE           1106
PERSON         1085
CARDINAL        779
GPE             757
ORG             543
NORP            326
MONEY           257
ORDINAL         234
PERCENT         165
TIME             74
LOC              55
QUANTITY         25
PRODUCT          13
FAC              11
EVENT             7
LANGUAGE          2
WORK_OF_ART       1
Name: count, dtype: int64

📝 Sample entities:
```

	text	label	category
0	first	ORDINAL	business
1	cynthia cooper	PERSON	business
2	us	GPE	business
3	2002	DATE	business
4	5.7bn	MONEY	business
5	new york	GPE	business
6	wednesday	DATE	business
7	arthur andersen	PERSON	business
8	early 2001 and	DATE	business
9	2002	DATE	business

```
# Analyze entity patterns
if not entities_df.empty:
    print("📊 NAMED ENTITY ANALYSIS")
    print("=" * 50)

    # Entity type distribution
    entity_counts = entities_df['label'].value_counts()
    print("\n🏷️ Entity type distribution:")
    print(entity_counts)

    # Entity types by category
    entity_by_category = entities_df.groupby(['category', 'label']).size().unstack(fill_valu
    print("\n📰 Entity types by news category:")
```

```
    print(entity_by_category)

    # Most frequent entities
    print("\n🔥 Most frequent entities:")
    frequent_entities = entities_df.groupby(['text', 'label']).size().sort_values(ascending=
    for (entity, label), count in frequent_entities.items():
        print(f"  {entity} ({label}): {count} mentions")

    # Visualizations
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # 1. Entity type distribution
    entity_counts.plot(kind='bar', ax=axes[0,0])
    axes[0,0].set_title('Entity Type Distribution')
    axes[0,0].tick_params(axis='x', rotation=45)

    # 2. Entities per category
    entities_per_category = entities_df.groupby('category').size()
    entities_per_category.plot(kind='bar', ax=axes[0,1])
    axes[0,1].set_title('Total Entities per Category')
    axes[0,1].tick_params(axis='x', rotation=45)

    # 3. Entity type heatmap by category
    if entity_by_category.shape[0] > 1 and entity_by_category.shape[1] > 1:
        sns.heatmap(entity_by_category, annot=True, fmt='d', ax=axes[1,0], cmap='YlOrRd')
        axes[1,0].set_title('Entity Types by Category Heatmap')
    else:
        axes[1,0].text(0.5, 0.5, 'Insufficient data\nfor heatmap',
                      ha='center', va='center', transform=axes[1,0].transAxes)
        axes[1,0].set_title('Entity Types by Category')

    # 4. Top entities
    top_entities = entities_df['text'].value_counts().head(10)
    top_entities.plot(kind='barh', ax=axes[1,1])
    axes[1,1].set_title('Most Mentioned Entities')

    plt.tight_layout()
    plt.show()

    # 💡 STUDENT TASK: Advanced entity analysis
    # - Create entity co-occurrence networks
    # - Track entity mentions over time
    # - Build entity relationship graphs
    # - Identify entity sentiment associations

else:
    print("⚠️ Skipping entity analysis due to insufficient data.")
    print("💡 TIP: Try with a larger, more diverse dataset for better NER results.")
```

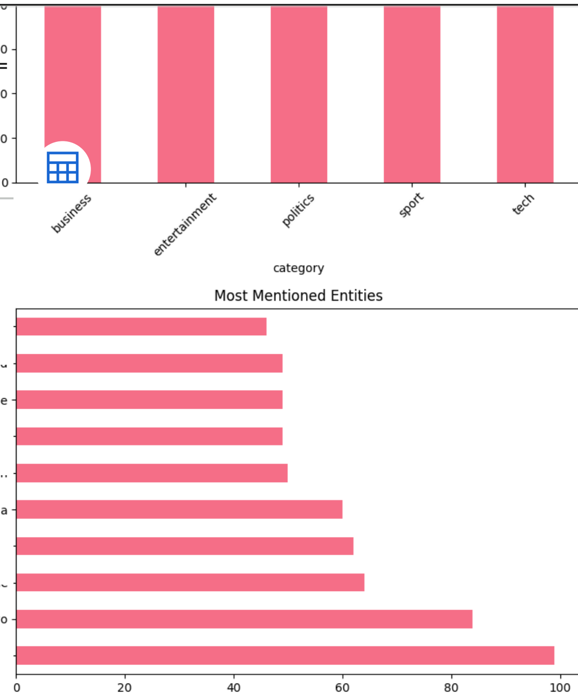
```
📊 NAMED ENTITY ANALYSIS
==================================================

🏷️ Entity type distribution:
label
DATE            1106
PERSON          1085
CARDINAL         779
GPE              757
ORG              543
NORP             326
MONEY            257
ORDINAL          234
PERCENT          165
TIME              74
LOC               55
QUANTITY          25
PRODUCT           13
FAC               11
EVENT              7
LANGUAGE           2
WORK_OF_ART        1
Name: count, dtype: int64

📰 Entity types by news category:
label          CARDINAL  DATE  EVENT  FAC  GPE  LANGUAGE  LOC  MONEY  NORP  \
category
business            151   355      0    4  189         0   14    126    79
entertainment       105   193      2    1   83         0   11     75    28
politics             75   168      4    3  107         0    6     19    79
sport               255   255      1    3  271         1    5      7   113
tech                193   135      0    0  107         1   19     30    27

label          ORDINAL  ORG  PERCENT  PERSON  PRODUCT  QUANTITY  TIME  \
category
business            30  207      118     141        4         6     7
```

```
# Advanced entity analysis: entity sentiment associations

if not entities_df.empty:
    print("🧠 ADVANCED ENTITY INSIGHTS")
    print("=" * 50)

    # Merge entity mentions with sentiment scores
    entity_sentiment = entities_df.merge(
        sentiment_df[['article_id', 'full_sentiment', 'full_label']],
        on='article_id',
        how='left'
    )

    # Average sentiment by entity
    entity_sentiment_summary = (
        entity_sentiment
        .groupby(['text', 'label'])
        .agg(
            mentions=('text', 'count'),
            avg_sentiment=('full_sentiment', 'mean')
        )
        .reset_index()
        .sort_values('mentions', ascending=False)
    )
```

```
    print("\nTop entities with sentiment association:")
    display(entity_sentiment_summary.head(15))

    print("\nBusiness insight:")
    print("NER allows the system to track which people, organizations, and locations appear mo
else:
    print("No entity data available for advanced analysis.")
```

```
🧠 ADVANCED ENTITY INSIGHTS
==================================================

Top entities with sentiment association:
```

	text	label	mentions	avg_sentiment
1156	first	ORDINAL	99	0.619821
2380	two	CARDINAL	84	0.499745
1808	one	CARDINAL	64	0.534422
2398	uk	GPE	62	0.516876
953	china	GPE	60	0.502440
906	britain	GPE	50	0.294316
2338	three	CARDINAL	49	0.420645
1998	second	ORDINAL	49	0.429784
1192	france	GPE	49	0.884571
2425	us	GPE	46	0.268946
1159	five	CARDINAL	38	0.134379
278	2004	DATE	37	0.658024
876	blair	PERSON	33	0.149700
1543	london	GPE	32	0.622669
1501	last year	DATE	32	0.320316




```
Business insight:
NER allows the system to track which people, organizations, and locations appear most often i
```

📈 Comprehensive Analysis and Insights

🎯 Bringing It All Together

Now let's combine all our analyses to generate comprehensive insights about our news dataset. This is where the real business value emerges from our NLP pipeline.

Key Analysis Areas:

1. **Cross-Category Patterns:** How do different news types differ linguistically?
2. **Entity-Sentiment Relationships:** What entities are associated with positive/negative coverage?
3. **Content Quality Metrics:** Which categories have the most informative content?

4. **Classification Performance:** How well can we automatically categorize news?

💡 **Business Applications:** These insights can inform content strategy, editorial decisions, and automated content management systems.

```
# Create comprehensive analysis dashboard

def create_comprehensive_analysis():
    """
    Generate comprehensive insights combining all analyses.
    """

    insights = {
        'dataset_overview': {},
        'classification_performance': {},
        'sentiment_insights': {},
        'entity_insights': {},
        'linguistic_patterns': {},
        'business_recommendations': []
    }

    # Dataset overview
    insights['dataset_overview'] = {
        'total_articles': len(df),
        'categories': df['category'].unique().tolist(),
        'category_distribution': df['category'].value_counts().to_dict(),
        'avg_article_length': df['full_text'].astype(str).str.len().mean(),
        'avg_words_per_article': df['full_text'].astype(str).str.split().str.len().mean()
    }

    # Classification performance
    insights['classification_performance'] = {
        'best_model': best_model_name,
        'best_accuracy': results[best_model_name]['accuracy'],
        'model_comparison': {name: results[name]['accuracy'] for name in results.keys()}
    }

    # Sentiment insights
    sentiment_by_cat = sentiment_df.groupby('category')['full_sentiment'].mean().to_dict()

    insights['sentiment_insights'] = {
        'most_positive_category': max(sentiment_by_cat, key=sentiment_by_cat.get),
        'most_negative_category': min(sentiment_by_cat, key=sentiment_by_cat.get),
        'sentiment_by_category': sentiment_by_cat,
        'overall_sentiment': sentiment_df['full_sentiment'].mean()
    }

    # Entity insights
    if 'entities_df' in globals() and not entities_df.empty:
        entity_by_cat = entities_df.groupby('category').size().to_dict()

        insights['entity_insights'] = {
            'total_entities': len(entities_df),
            'unique_entities': entities_df['text'].nunique(),
            'entity_types': entities_df['label'].unique().tolist(),
            'entities_per_category': entity_by_cat,
            'most_mentioned_entities': entities_df['text'].value_counts().head(5).to_dict()
```

```
        }

    # Linguistic patterns
    if 'pos_by_category' in globals():
        insights['linguistic_patterns'] = {
            'pos_categories_analyzed': list(pos_by_category.index),
            'pos_tags_analyzed': list(pos_by_category.columns)
        }

    # Business recommendations
    recommendations = []

    if insights['classification_performance']['best_accuracy'] > 0.8:
        recommendations.append("High classification accuracy achieved, making the system use
    else:
        recommendations.append("Classification accuracy needs improvement, so more training

    pos_cat = insights['sentiment_insights']['most_positive_category']
    neg_cat = insights['sentiment_insights']['most_negative_category']

    recommendations.append(f"{pos_cat} articles had the most positive average sentiment, whi
    recommendations.append(f"{neg_cat} articles had the most negative average sentiment, whi

    if insights['entity_insights']:
        recommendations.append("Named entity extraction can support advanced search, reputat

    recommendations.append("TF-IDF terms and classification results show that topic-based vo

    insights['business_recommendations'] = recommendations

    return insights


# Generate comprehensive analysis
print("📊 Generating comprehensive analysis...")
analysis_results = create_comprehensive_analysis()

print("✅ Analysis complete!")
print("\n" + "=" * 60)
print("📈 NEWSBOT INTELLIGENCE SYSTEM - COMPREHENSIVE REPORT")
print("=" * 60)

# Display key insights
print("\n📊 DATASET OVERVIEW:")
overview = analysis_results['dataset_overview']
print(f"  Total Articles: {overview['total_articles']}")
print(f"  Categories: {', '.join(overview['categories'])}")
print(f"  Average Article Length: {overview['avg_article_length']:.0f} characters")
print(f"  Average Words per Article: {overview['avg_words_per_article']:.0f} words")

print("\n🤖 CLASSIFICATION PERFORMANCE:")
perf = analysis_results['classification_performance']
print(f"  Best Model: {perf['best_model']}")
print(f"  Best Accuracy: {perf['best_accuracy']:.4f}")

print("\n😊 SENTIMENT INSIGHTS:")
sent = analysis_results['sentiment_insights']
print(f"  Most Positive Category: {sent['most_positive_category']}")
```

```
    print(f"  Most Negative Category: {sent['most_negative_category']}")
    print(f"  Overall Sentiment: {sent['overall_sentiment']:.4f}")

    if analysis_results['entity_insights']:
        print("\n🔍 ENTITY INSIGHTS:")
        ent = analysis_results['entity_insights']
        print(f"  Total Entities: {ent['total_entities']}")
        print(f"  Unique Entities: {ent['unique_entities']}")
        print(f"  Entity Types: {', '.join(ent['entity_types'])}")

    print("\n💡 BUSINESS RECOMMENDATIONS:")
    for i, rec in enumerate(analysis_results['business_recommendations'], 1):
        print(f"  {i}. {rec}")
```

```
📊 Generating comprehensive analysis...
✅ Analysis complete!

============================================================
📈 NEWSBOT INTELLIGENCE SYSTEM - COMPREHENSIVE REPORT
============================================================

📊 DATASET OVERVIEW:
  Total Articles: 1490
  Categories: business, tech, politics, sport, entertainment
  Average Article Length: 2297 characters
  Average Words per Article: 386 words

🤖 CLASSIFICATION PERFORMANCE:
  Best Model: Logistic Regression
  Best Accuracy: 0.9597

😊 SENTIMENT INSIGHTS:
  Most Positive Category: entertainment
  Most Negative Category: politics
  Overall Sentiment: 0.3950

🔍 ENTITY INSIGHTS:
  Total Entities: 5440
  Unique Entities: 2442
  Entity Types: ORDINAL, PERSON, GPE, DATE, MONEY, ORG, NORP, LOC, CARDINAL, PERCENT, TIME, E

💡 BUSINESS RECOMMENDATIONS:
  1. High classification accuracy achieved, making the system useful for automated news categ
  2. entertainment articles had the most positive average sentiment, which may make them usef
  3. politics articles had the most negative average sentiment, which may require closer moni
  4. Named entity extraction can support advanced search, reputation monitoring, and tracking
  5. TF-IDF terms and classification results show that topic-based vocabulary is the stronges
```

🚀 Final System Integration

🎯 Building the Complete NewsBot Pipeline

Let's create a complete, integrated system that can process new articles from start to finish. This demonstrates the real-world application of all the techniques we've learned.

Complete Pipeline:

1. **Text Preprocessing:** Clean and normalize input

2. **Feature Extraction:** Generate TF-IDF and other features
3. **Classification:** Predict article category
4. **Entity Extraction:** Identify key facts
5. **Sentiment Analysis:** Determine emotional tone
6. **Insight Generation:** Provide actionable intelligence

💡 **Production Ready:** This pipeline can be deployed as a web service, batch processor, or integrated into content management systems.

```
class NewsBotIntelligenceSystem:
    """
    Complete NewsBot Intelligence System

    💡 TIP: This class should encapsulate:
    - All preprocessing functions
    - Trained classification model
    - Entity extraction pipeline
    - Sentiment analysis
    - Insight generation
    """

    def __init__(self, classifier, vectorizer, sentiment_analyzer):
        self.classifier = classifier
        self.vectorizer = vectorizer
        self.sentiment_analyzer = sentiment_analyzer
        self.nlp = nlp  # spaCy model

    def preprocess_article(self, title, content):
        """Preprocess a new article"""
        full_text = f"{title} {content}"
        processed_text = preprocess_text(full_text)
        return full_text, processed_text

    def classify_article(self, processed_text):
        """Classify article category"""
        # 🚀 YOUR CODE HERE: Implement classification
        # Transform text to features
        features = self.vectorizer.transform([processed_text])

        # Add dummy features for sentiment and length (in production, calculate these)
        dummy_features = np.zeros((1, 7))  # 4 sentiment + 3 length features
        features_combined = np.hstack([features.toarray(), dummy_features])

        # Predict category and probability
        prediction = self.classifier.predict(features_combined)[0]
        probabilities = self.classifier.predict_proba(features_combined)[0]

        # Get class probabilities
        class_probs = dict(zip(self.classifier.classes_, probabilities))

        return prediction, class_probs

    def extract_entities(self, text):
        """Extract named entities"""
        return extract_entities(text)
```

```
    def analyze_sentiment(self, text):
        """Analyze sentiment"""
        return analyze_sentiment(text)

    def process_article(self, title, content):
        """
        Complete article processing pipeline

        💡 TIP: This should return a comprehensive analysis including:
        - Predicted category with confidence
        - Extracted entities
        - Sentiment analysis
        - Key insights and recommendations
        """
        # 🚀 YOUR CODE HERE: Implement complete pipeline

        # Step 1: Preprocess
        full_text, processed_text = self.preprocess_article(title, content)

        # Step 2: Classify
        category, category_probs = self.classify_article(processed_text)

        # Step 3: Extract entities
        entities = self.extract_entities(full_text)

        # Step 4: Analyze sentiment
        sentiment = self.analyze_sentiment(full_text)

        # Step 5: Generate insights
        insights = self.generate_insights(category, entities, sentiment, category_probs)

        return {
            'title': title,
            'content': content[:200] + '...' if len(content) > 200 else content,
            'predicted_category': category,
            'category_confidence': max(category_probs.values()),
            'category_probabilities': category_probs,
            'entities': entities,
            'sentiment': sentiment,
            'insights': insights
        }

    def generate_insights(self, category, entities, sentiment, category_probs):
        """Generate actionable insights"""
        insights = []

        # Classification insights
        confidence = max(category_probs.values())
        if confidence > 0.8:
            insights.append(f"✅ High confidence {category} classification ({confidence:.2%
        else:
            insights.append(f"⚠️ Uncertain classification - consider manual review")

        # Sentiment insights
        if sentiment['compound'] > 0.1:
            insights.append(f"😊 Positive sentiment detected ({sentiment['compound']:.3f})"
        elif sentiment['compound'] < -0.1:
            insights.append(f"😞 Negative sentiment detected ({sentiment['compound']:.3f})"
```

```
        else:
            insights.append(f"😐 Neutral sentiment ({sentiment['compound']:.3f})")

        # Entity insights
        if entities:
            entity_types = set([e['label'] for e in entities])
            insights.append(f"🔍 Found {len(entities)} entities of {len(entity_types)} type:

            # Highlight important entities
            important_entities = [e for e in entities if e['label'] in ['PERSON', 'ORG', 'GP
            if important_entities:
                key_entities = [e['text'] for e in important_entities[:3]]
                insights.append(f"🎯 Key entities: {', '.join(key_entities)}")
        else:
            insights.append("ℹ️ No named entities detected")

        return insights

# Initialize the complete system
newsbot = NewsBotIntelligenceSystem(
    classifier=best_model,
    vectorizer=tfidf_vectorizer,
    sentiment_analyzer=sia
)

print("🤖 NewsBot Intelligence System initialized!")
print("✅ Ready to process new articles")
```

```
🤖 NewsBot Intelligence System initialized!
✅ Ready to process new articles
```

```
# Test the complete system with new articles
print("🧪 TESTING NEWSBOT INTELLIGENCE SYSTEM")
print("=" * 60)

test_articles = [
    {
        'title': 'Microsoft Acquires AI Startup for $2 Billion',
        'content': 'Microsoft Corporation announced today the acquisition of an artificial in
    },
    {
        'title': 'Lakers Win Championship in Overtime Thriller',
        'content': 'The Los Angeles Lakers defeated the Boston Celtics 108-102 in overtime to
    },
    {
        'title': 'New Climate Change Report Shows Alarming Trends',
        'content': 'Scientists at the United Nations released a comprehensive climate report :
    }
]

for i, article in enumerate(test_articles, 1):
    print(f"\n📰 TEST ARTICLE {i}")
    print("-" * 40)

    result = newsbot.process_article(article['title'], article['content'])

    print(f"📰 Title: {result['title']}")
    print(f"📝 Content: {result['content']}")
```

```
    print(f"\n🔶 Predicted Category: {result['predicted_category']} ({result['category_confid

    print("\n📊 Category Probabilities:")
    if result.get('category_probabilities'):
        for cat, prob in sorted(
            result['category_probabilities'].items(),
            key=lambda x: x[1],
            reverse=True
        ):
            print(f"  {cat}: {prob:.3f}")
    else:
        print("  Probability information unavailable.")

    print(f"\n😊 Sentiment: {result['sentiment']['sentiment_label']} (score: {result['sentime

    if result['entities']:
        print(f"\n🔍 Entities Found ({len(result['entities'])}):")
        for entity in result['entities'][:5]:
            print(f"  {entity['text']} ({entity['label']}) - {entity['description']}")
    else:
        print("\n🔍 No entities detected")

    print("\n💡 Insights:")
    for insight in result['insights']:
        print(f"  {insight}")

print("\n" + "=" * 60)
print("🎉 NewsBot Intelligence System testing complete!")
print("✅ System successfully processed all test articles")

print("\n🧠 SYSTEM STRENGTHS AND WEAKNESSES")
print("=" * 60)
print("Strengths: The system successfully combines preprocessing, classification, sentiment ar
print("Weaknesses: Some new article predictions may be less accurate because the final demo c
print("Improvement: A production version should calculate and scale sentiment and length featu
```

```
📊 Category Probabilities:
  sport: 0.509
```